

SF WORKSPACE

Technical Guide

Repository-verified architecture, modules, Salesforce integrations,
security, build, and operations

Version: 1.0.0 **Guide date:** 24 September 2026

Product: Salesforce Workspace (short name: SF Workspace)

Evidence basis: source tree, manifest, package metadata, tests, build scripts, README, and privacy policy in the supplied repository.

Classification key

Working: implemented and directly verifiable in code/tests. **Partial:** useful implementation with explicit gaps. **Not found:** requested concept has no module or route in this repository.

Executive summary

SF Workspace is a Manifest V3 Chrome extension for Salesforce administrators, developers, and testers. Its primary experience is a dedicated extension popup window opened from the toolbar; the same application is also declared as a Chrome side panel. It detects Salesforce context from tab URLs, connects either by validating the signed-in tab session or through OAuth 2.0 Authorization Code with PKCE, and then calls Salesforce directly from the extension.

The repository implements record search and controlled record changes, user and access inspection, local Flow/deployment/debug-log analysis, a read-only SOQL workspace, metadata exploration, dependency lookup, field-mapping checks, test-evidence capture, org switching, and themes. There is no content script and no page DOM scraping. There is also no general Code & Automation Explainer, no Apex class source explainer, no Flow-definition explainer, and no trigger source explainer.

Most data processing is local. Credentials are separated from durable data: access tokens and browser-session SIDs use **chrome.storage.session**; an OAuth refresh token is persisted to **chrome.storage.local** only when the user enables "Keep OAuth sign-in." Metadata and test screenshots use IndexedDB. Record drafts use the workspace window's sessionStorage.

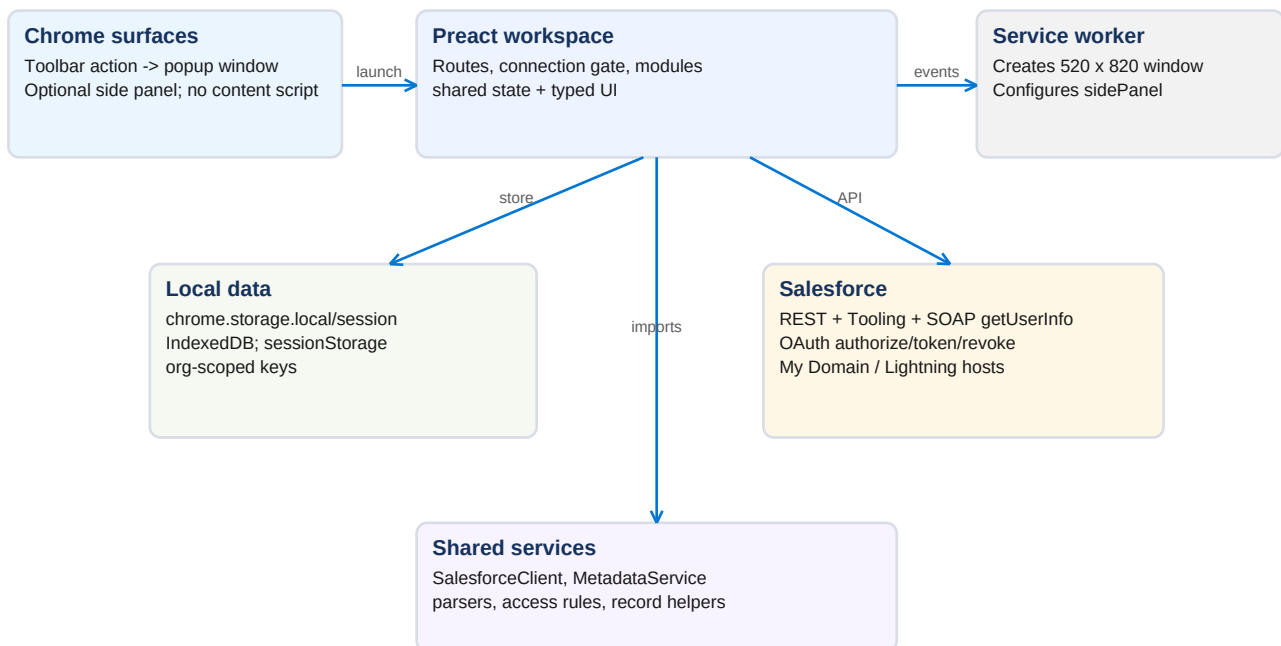
Repository verification snapshot

- Package and built manifest version: 1.0.0 (src/manifest.json is stamped from package.json during build).
- 145 tests across 13 files passed on 24 September 2026.
- Production build completed and emitted dist/sidepanel.js and dist/background.js.
- The aggregate npm run check did not complete because ESLint reports five browser-global/no-unused-variable errors in src/sidepanel/theme-boot.js. Type checking passed before that failure.

Feature and module inventory

Area	Tool/module	Status	Where it runs	Verified capability
Home	Overview	Working	Workspace UI	Page/org context, recent tools, saved items, Setup links, org/user summary
Records	Search	Working	Workspace UI	ID lookup, SOSL, object filtering, RecentlyViewed
Records	Details/forms	Working	Workspace UI	Layout-aware fields, lookups, related lists, create/edit/delete with checks
Users	Users	Working	Workspace UI	Filtered/paged users and assigned-access detail
Access	Access Explainer	Working	Workspace UI	Object, field, and record verdicts with confidence
Flows & Forms	Flow error translator	Working	Workspace UI / local	Rule-based pasted-error extraction and guidance
Flows & Forms	Form tester	Working	Workspace UI / local	Steps, screenshots, local sessions, HTML/Markdown export
Development	SOQL workspace	Working	Workspace UI	Read-only REST/Tooling queries, paging, sorting, CSV, history
Development	Debug logs	Working	Workspace UI	Tooling ApexLog browsing or pasted logs; local parser
Development	Deployment errors	Working	Workspace UI / local	Parses sf/sfdx JSON or text; failures, tests, coverage
Metadata	Objects & fields	Working	Workspace UI	Global/object describe, cache, field relationships/access flags
Metadata	Field mapping	Partial	Workspace UI	Static metadata compatibility checks only
Metadata	Dependencies	Partial	Workspace UI	Beta Tooling dependency graph for supported custom metadata
Settings	Connection/org/theme/privacy	Working	Workspace UI	Auth, org profiles/switching, theme/accent, storage controls
Background	Service worker	Working	MV3 worker	Opens popup window; configures side-panel behavior
Content script	None	Not found	N/A	No content_scripts entry; no Salesforce DOM access
Code explainer	General/Apex/trigger/Flow source	Not found	N/A	No route, UI, parser, Metadata API retrieval, or source analysis

Architecture



Surface and runtime model

- The manifest declares a side_panel at sidepanel.html and a module background service worker. The toolbar action itself has no popup; the service worker handles action clicks and creates a 520 x 820 Chrome popup window containing sidepanel.html.
- The same Preact application runs in the dedicated extension window and the optional side panel. There is no distinct full-page route and no content script.
- All feature modules execute in the extension page. The service worker does not proxy Salesforce traffic or retain application state.

```
src/manifest.json - lines 1-46
src/background/service-worker.ts - lines 1-28
src/sidepanel/App.tsx - lines 25-40, 209-228
```

Internal layering

- Presentation: App, shared components, routes, hooks, and 14 tool components under src/sidepanel.
- State/orchestration: WorkspaceProvider binds active-tab context, selected org, connection state, API client, metadata service, theme, recent tools, and navigation.
- Domain services: records, users, access, metadata, dependencies, parsers, mapping, themes, favorites, test sessions, security redaction.
- Persistence adapters: chrome.storage.local/session and IndexedDB behind KVStore/OrgScopedStore; sessionStorage is used only for unsaved record drafts.
- External boundary: SalesforceClient attaches Bearer authorization, confines requests to the connected instance URL, caps concurrency at four, retries selected GET failures, refreshes once on 401, and tracks Sforce-Limit-Info.

```
src/sidepanel/state/workspace.tsx - lines 39-80, 214-328, 414-449
src/shared/api/client.ts - lines 103-181, 183-281
```

Technology stack

Technology/API	Why it is used	Primary evidence
TypeScript 6 / TSX	Strict typed implementation for UI and Salesforce domain models.	package.json; tsconfig.json
Preact 10	Compact component and hooks layer for the extension UI.	src/sidepanel/main.tsx; App.tsx
Chrome Manifest V3	Extension packaging, module service worker, action, commands, CSP.	src/manifest.json
Chrome sidePanel/windows/tabs	Optional side panel, dedicated popup, source-tab tracking and org safety.	service-worker.ts; usePageContext.ts
Chrome cookies	Reads Salesforce sid candidates from the launching tab cookie store.	tabSessionAuth.ts:45-89
Chrome identity	OAuth PKCE redirect via launchWebAuthFlow.	services/auth.ts:30-80
chrome.storage + IndexedDB	Settings/profiles/tokens and larger org-scoped caches/screenshot data.	platform.ts; storage/kv.ts
Fetch + Salesforce REST	Versions, limits, query, describe, sObject CRUD/search.	shared/api/client.ts
Salesforce Tooling API	ApexLog, Tooling SOQL, CustomObject/CustomField and dependency queries.	DebugLogs.tsx; dependencies.ts
Salesforce SOAP Partner getUserInfo	Fallback identity validation for browser SIDs without OpenID scope.	tabSessionAuth.ts:28-42
esbuild	Bundles two ESM entry points; stamps manifest version; no remote code.	scripts/build.mjs
Vitest / ESLint	Unit tests and static quality gates.	vitest.config.ts; eslint.config.js
CSS + generated theme tokens	Responsive layout, light/dark/contrast and accessible accents.	styles.css; shared/theme/accent.ts

Home / Overview

Working

Purpose and user outcome

Gives nontechnical users an org-and-page dashboard: current context, production warning, recent tools, saved objects/records/queries, Setup shortcuts, and org/user/API usage details.

User-to-result flow

- Workspace derives the source tab and org identity from the URL, loads stored org profiles, then optionally connects.
- Overview links directly to Record details or Objects & fields when page context includes an object/record.
- Saved items are loaded from the selected org's profile-scoped store; opening an item navigates internally.
- When connected, loadOrgInfo runs independent REST queries for Organization and User, asks /limits, and reads API versions; unavailable sections degrade independently.

Technologies and APIs

Preact hooks; active-tab URL parsing; chrome.storage; Salesforce REST Query, Limits, and Versions APIs; generated Lightning Setup URLs.

Main files and dependencies

```
src/sidepanel/modules/overview/Overview.tsx - lines 24-346
src/shared/salesforce/orgInfo.ts - lines 1-75
src/shared/workspace/favorites.ts - lines 1-65
src/shared/salesforce/links.ts - lines 1-56
```

Loading, errors, permissions, authentication, and data

Skeleton/loading and retryable error states come from useAsync and shared state components. Saved items are per org. The home page itself can render offline, but org details require a valid connection. Production is derived from host/API identity and can be overridden in settings.

Verified limits

Setup shortcuts are URL builders rather than Setup API checks; visibility and authorization remain Salesforce's responsibility.

Records: search, navigation, details, and changes

Working

Purpose and user outcome

Finds records by Salesforce ID, SOSL text, an optional object, or RecentlyViewed; then displays layout-aware readable fields, highlights, parent links, and queryable child relationships. Authorized users can create, edit, and delete when the global setting permits it.

User-to-result flow

- Search validates an ID or creates a SOSL FIND expression; unfiltered SOSL targets a fixed common-object set. Recently viewed records come from a SOQL query on RecentlyViewed.
- Opening a result resolves the object from the ID prefix when needed, loads object describe metadata, then calls the REST sObject record endpoint. Returned fields therefore reflect FLS; missing described fields are counted as hidden/unreturned.
- Lookup display names are resolved through relationship SOQL where possible. Layout and compact-layout endpoints organize fields and highlights; metadata grouping is the fallback.
- Create/edit forms derive writable controls from describe metadata, validate locally, preserve drafts in sessionStorage, then POST or PATCH the sObject endpoint. Edit rechecks LastModifiedDate and sends If-Unmodified-Since unless the user accepts overwrite.
- Delete first checks live UserRecordAccess, requires confirmation (typing DELETE in production), and issues REST DELETE with an optional concurrency header.

Technologies and APIs

Salesforce REST sObject, Query, Search/SOSL, describe/global describe, describe/layouts, compactLayouts/primary, UserRecordAccess; Preact forms; sessionStorage; org-scoped favorites.

Main files and dependencies

```
src/sidepanel/modules/records/RecordSearch.tsx - lines 37-190
src/sidepanel/modules/records/RecordNavigator.tsx - lines 33-630
src/sidepanel/modules/records/RecordForm.tsx - lines 63-83, 236-559
src/shared/salesforce/records.ts - lines 36-235
src/shared/records/recordEdit.ts - lines 1-313
src/shared/api/client.ts - lines 235-280
```

Loading, errors, permissions, authentication, and data

ConnectionGate blocks API-backed views. AbortSignals cancel superseded loads. Errors are normalized and field errors are mapped without dropping edits. Describe metadata is cached per org. Drafts are window-session-only; favorites and history are durable per org. CRUD is disabled when Settings turns off record changes and always remains subject to Salesforce permissions.

Verified limits

Record-type picklists, lookup filters, and layout-required fields are not prevalidated; Salesforce can reject them on save. Rich text, files, encrypted fields, and compound address/location fields are not directly editable. SOSL indexing can lag. Related lists are inferred from describe relationships, not rendered from Salesforce's page layout.

Users

Working

Purpose and user outcome

Provides a searchable, filterable list of users and a detail view for identity, profile, permission-set groups, permission sets, permission-set licenses, selected system permissions, and direct public-group/queue membership.

User-to-result flow

- The list constructs SOQL with safe escaped search input, filters and sorting, requests 25 rows, and obtains a count separately.
- Opening a user starts independent detail queries so readable sections can render even if setup-level objects are denied.
- The “Check this user’s access” action routes the chosen user into Access Explainer.

Technologies and APIs

Salesforce REST Query API over User, Profile, PermissionSetAssignment, PermissionSetLicenseAssign, GroupMember and Group; local pagination state.

Main files and dependencies

`src/sidepanel/modules/users/Users.tsx` - lines 48-396

`src/shared/salesforce/users.ts` - lines 1-268

Loading, errors, permissions, authentication, and data

Search is debounced. Each detail part carries its own loading/error result. Setup-related sections may require View Setup and Configuration. No passwords or login-as action exist. Results are not persisted beyond UI state.

Verified limits

SOQL OFFSET constrains practical paging to the first 2,000 matches. Membership is direct only; roles, territories, and nested group expansion are not calculated. “Assigned access” is intentionally not presented as effective access.

Access Explainer

Working

Purpose and user outcome

Answers a scoped question about one user and object, optionally one field and one record. It reports Yes/No/Unknown with Confirmed/Likely/Uncertain labels and names the profile, permission set, or group assignment that supplies a grant when visible.

User-to-result flow

- User search queries User; object/field choices come from describe metadata; record IDs are validated and normalized.
- collectAccessInput loads PermissionSetAssignment sources, ObjectPermissions, selected PermissionSet system permissions, FieldPermissions, and optionally UserRecordAccess. Independent queries are wrapped so one denied query yields Unknown instead of collapsing the result.
- interpretAccess applies conservative local rules: missing data never becomes a denial; session-activated grants are “likely”; field access never implies record access; exact record conclusions come only from UserRecordAccess.
- The UI renders separate object, field, record, and assignment sections plus explicit notes.

Technologies and APIs

Salesforce REST Query API; standard setup/access objects; describe metadata; deterministic TypeScript interpretation rules.

Main files and dependencies

src/sidepanel/modules/access/AccessExplainer.tsx - lines 46-301

src/shared/access/queries.ts - lines 21-190

src/shared/access/interpret.ts - lines 112-254

Loading, errors, permissions, authentication, and data

Requires a connection and whatever setup visibility the connected user has. Queries support cancellation and degrade independently. Nothing is stored. UI errors remain retryable.

Verified limits

It does not simulate arbitrary sharing-rule evaluation, Apex runtime context, restriction/scoping rules, or all license/feature constraints. Record results are strongest only when a record ID is supplied and UserRecordAccess is queryable.

Flows & Forms

Working for both implemented tools

Purpose and user outcome

Flow Error Translator explains pasted runtime/fault text with local rules. Form Tester records repeatable test steps, expected/actual results, statuses, sanitized URLs, and optional screenshots; it exports HTML or Markdown reports.

User-to-result flow

- Translator parses the pasted string locally, extracts flow/element/interview/error/user/org/record details, matches known patterns, and displays causes and verification steps. Copy output is redacted. It does not call Salesforce or an AI service.
- Form Tester creates an org-scoped (or unscoped local) test session in IndexedDB. A tester edits steps and may explicitly capture the visible tab as a JPEG through `chrome.tabs.captureVisibleTab`, or attach an image.
- Exports load selected screenshot records, sanitize URLs, redact secrets and optionally emails, then download generated HTML or Markdown using a Blob URL.

Technologies and APIs

Local TypeScript parsers; Preact; IndexedDB; `activeTab/tabs.captureVisibleTab`; Blob downloads; security redaction helpers.

Main files and dependencies

```
src/sidepanel/modules/flows/FlowErrorTranslator.tsx - lines 24-158
src/shared/parsers/flowError.ts - lines 1-362
src/sidepanel/modules/flows/FormTester.tsx - lines 35-520
src/shared/testing/formSession.ts - lines 1-188
src/shared/security/redact.ts - lines 1-47
```

Loading, errors, permissions, authentication, and data

Neither tool requires Salesforce API authentication for its core local mode. Test sessions/screenshots are stored locally and segregated by org when an org is known. Capture errors are displayed and users can upload instead. Screenshots are capped at 6 MB in the UI. Export options control URLs, screenshots, and email redaction.

Verified limits

This is not a Flow-definition inspector, builder, debugger, or deployment tool. Explanations are heuristic and depend on the pasted message. Screenshot permission depends on an `activeTab` grant initiated by the extension action/shortcut.

Development: SOQL workspace

Working

Purpose and user outcome

Runs read-only SOQL against either the standard Query API or Tooling Query API, displays flattened results, supports client-side filter/sort, pagination, cancellation, saved queries/history, and CSV export.

User-to-result flow

- A local validator accepts only SELECT/FIND-style read operations and rejects mutation-oriented text.
- Run sends the query to /query or /tooling/query. The first page displays immediately; Load more or Load all follows nextRecordsUrl.
- Pagination stops if known daily usage exceeds 90% and Load all caps at 10,000 records. Results are flattened for tables/CSV; Salesforce IDs can route to Record details.
- History (20 entries) and favorites are saved in the selected org's chrome.storage.local namespace.

Technologies and APIs

Salesforce REST Query and Tooling Query APIs; AbortController; Sforce-Limit-Info tracking; local CSV generation and Blob download.

Main files and dependencies

```
src/sidepanel/modules/development/SoqlWorkspace.tsx - lines 20-281
src/shared/parsers/soql.ts - lines 1-91
src/shared/api/client.ts - lines 193-233
```

Loading, errors, permissions, authentication, and data

ConnectionGate requires authentication. Users can cancel in-flight queries. API and query errors use shared explanations. Result rows themselves are not persisted; only query text history/favorites are stored.

Verified limits

The client-side validation is a guardrail, not a SQL parser or Salesforce authorization layer. Queries still execute with the connected user's permissions. A Tooling query can read metadata-oriented objects but cannot deploy changes.

Development: Debug logs and deployment errors

Working

Purpose and user outcome

Debug Logs browses recent ApexLog rows (or accepts a pasted log), downloads and locally parses log bodies for exceptions, governor limits, SOQL, DML, and events. Deployment Errors parses pasted sf/sfdx deployment text or JSON into component, test, and coverage findings.

User-to-result flow

- Org log mode runs Tooling SOQL on ApexLog (up to 50, optionally current user), then fetches `/tooling/subjects/ApexLog/{id}/Body`.
- The debug parser identifies events and summaries locally; filters cap the displayed event table at 1,000 while retaining the raw-log download.
- Deployment input is classified as JSON or text, normalized into component/test/coverage categories, and rendered with likely next steps. A redacted summary can be copied.

Technologies and APIs

Salesforce Tooling API; local deterministic parsers; Blob download; redaction. Deployment analysis requires no org connection.

Main files and dependencies

```
src/sidepanel/modules/development/DebugLogs.tsx - lines 17-291
src/shared/parsers/debugLog.ts - lines 1-159
src/sidepanel/modules/development/DeployAssistant.tsx - lines 8-168
src/shared/parsers/deployErrors.ts - lines 1-508
```

Loading, errors, permissions, authentication, and data

Org log browsing uses ConnectionGate and shared loading/retry states; paste modes operate offline. The extension deliberately does not create trace flags because that would change org configuration. Pasted content is held in component memory, not durable storage.

Verified limits

These tools explain logs/errors, not Apex source. They do not retrieve ApexClass/ApexTrigger bodies, build symbol graphs, or reason about business intent. Deployment parser coverage is limited to recognized CLI shapes and heuristics.

Metadata

Working Object Explorer; **Partial** Mapping and Dependencies

Purpose and user outcome

Objects & Fields searches global describe data and shows field types, relationships, lengths, and create/update flags. Field Mapping compares two described fields for obvious compatibility problems. Dependencies shows “used by” and “uses” edges for supported custom objects/fields.

User-to-result flow

- MetadataService caches trimmed global/object describes per org with a configurable TTL; Refresh bypasses cache.
- Object Explorer filters/sorts fields and links to Object Manager; create is offered when describe says the object is createable.
- Mapping runs checkMapping on source and target field metadata for type, size, precision/scale, relationship, requiredness, and operation-specific writeability issues.
- Dependencies resolves CustomObject/CustomField Tooling IDs, then runs two MetadataComponentDependency queries with a 2,000-row cap.

Technologies and APIs

Salesforce REST describe, layout and compact-layout endpoints; Tooling Query API; IndexedDB metadata cache; deterministic mapping rules.

Main files and dependencies

```
src/sidepanel/modules/metadata/ObjectExplorer.tsx - lines 13-58
src/shared/salesforce/metadata.ts - lines 180-288
src/sidepanel/modules/metadata/FieldMapping.tsx - lines 7-35
src/shared/mapping/fieldMapping.ts - lines 1-180
src/sidepanel/modules/metadata/Dependencies.tsx - lines 8-36
src/shared/salesforce/dependencies.ts - lines 28-80
```

Loading, errors, permissions, authentication, and data

All three require authentication. Cached entries are org-scoped and expose cache age/source; errors are retryable. Clearing metadata removes cached descriptions without touching Salesforce.

Verified limits

Mapping does not inspect source data, validation rules, automations, or record-type UI constraints. MetadataComponentDependency is beta and can omit standard components, dynamic Apex/SOQL, reports, and managed-package internals; standard objects/fields generally cannot be resolved.

Settings, themes, connection, and org switching

Working

Purpose and user outcome

Controls connection behavior, global/per-org OAuth client IDs, remembered refresh tokens, org labels/colors/environment/API version, selected org, cache TTL, record-change enablement, export email redaction, and system/light/dark/high-contrast themes with accessible accent presets.

User-to-result flow

- Settings writes GlobalSettings and OrgProfile records to chrome.storage.local. Org profiles are discovered from Salesforce tab URLs and later enriched with verified org/user identity.
- The header follows the current Salesforce tab unless a user pins a stored org. The app warns when the pinned org differs from the tab. Selecting “Use this tab” repoints the dedicated window or activates the chosen tab in side-panel mode.
- Theme selection updates CSS custom properties and a small boot payload in localStorage to avoid startup flash. Accent generation adjusts readability and separates destructive red.
- Forget org clears tokens, session storage namespace, IndexedDB namespace, profile, cache, query history, favorites, and test sessions for that org.

Technologies and APIs

Chrome tabs/windows/storage; localStorage theme boot; IndexedDB; Salesforce identity/profile data; WCAG contrast calculations.

Main files and dependencies

```
src/sidepanel/modules/settings/Settings.tsx - lines 20-475
src/sidepanel/App.tsx - lines 42-101
src/sidepanel/state/workspace.tsx - lines 330-447
src/shared/org/profiles.ts - lines 30-157
src/shared/theme/accent.ts - lines 1-160
src/sidepanel/theme-boot.js - lines 1-18
```

Loading, errors, permissions, authentication, and data

Connection and org settings surface explicit errors and confirmations. Sensitive access tokens are not displayed. Forgetting an org is local and does not delete Salesforce data. Theme and settings changes propagate through chrome.storage change listeners.

Verified limits

The privacy policy still contains a publication placeholder requesting a publisher support contact/effective date. The source theme-boot.js currently causes the repository-wide lint task to fail because browser globals are not declared in its lint context.

Authentication and data flow

Preferred tab-session path

- The active/source Salesforce tab is parsed into a normalized OrgIdentity. A connection attempt obtains the exact Chrome cookie store containing that tab and searches only Salesforce sid cookies.
- Each candidate SID is validated against Salesforce. The extension tries /services/oauth2/userinfo and falls back to Partner SOAP getUserInfo; it also queries Organization and checks org/user ID prefixes and origin continuity.
- A validated token is tagged authSource=tab-session and sourceTabId, then stored only in chrome.storage.session. The client stops using it if the source tab closes, changes origin/org, or the session expires. Disconnect forgets a tab SID and does not revoke it, avoiding a Salesforce logout.

`src/sidepanel/services/tabSessionAuth.ts` - lines 19-90

OAuth PKCE fallback

- The user configures a public-client Consumer Key; no client secret is requested or stored.
- `chrome.identity.launchWebAuthFlow` opens Salesforce authorization with a random state, PKCE verifier/challenge, api plus refresh_token/offline_access scopes.
- The authorization code is exchanged at Salesforce's token endpoint. The returned instance URL must match the selected org or the token is revoked/discarded.
- Access tokens live in `chrome.storage.session`. A refresh token is copied to `chrome.storage.local` only when "Keep OAuth sign-in after Chrome restarts" is enabled. A Web Lock serializes refresh per org; terminal refresh errors clear tokens.

`src/shared/auth/oauth.ts` - lines 46-206

`src/sidepanel/services/auth.ts` - lines 30-132

`src/shared/auth/tokenStore.ts` - lines 1-48

Request controls

- Bearer credentials are sent with credentials: omit and only to URLs rooted at the connected instance URL.
- Maximum concurrent calls default to four. GETs retry twice on 502/503/504. A single refresh is attempted on 401.
- AbortSignals cancel work; Sforce-Limit-Info feeds UI usage and prevents large reads beyond a 90% threshold.
- API errors are typed and translated into actionable UI messages.

Permissions, privacy, and security

Manifest item	Verified use	Security note
sidePanel	Declares optional side-panel surface and configures action behavior.	Primary action opens a separate extension window.
storage	Settings, profiles, tokens, org-scoped favorites/history; IndexedDB is web storage.	Session/local separation limits durable credentials.
identity	OAuth redirect interception.	PKCE public-client flow; no client secret.
activeTab	User-initiated visible-tab screenshots.	Capture occurs only on explicit Form Tester action.
tabs	Read source-tab URL/state, enumerate candidate Salesforce tabs, activate/repoint.	No page DOM or content script.
cookies	Read Salesforce sid candidates from source tab's cookie store.	Candidates are validated; not displayed/exported/logged.
Salesforce hosts	REST/Tooling/SOAP/OAuth calls and context recognition.	Salesforce-only allowlist; client rejects cross-instance request URLs.
CSP	script-src self; object-src self; image self/data/blob.	No remote executable code; screenshots/data images remain possible.

Data inventory

- chrome.storage.local: global settings, org profiles, org-scoped favorites and SOQL history; OAuth refresh token only with opt-in.
- chrome.storage.session: access tokens/SIDs and session-scoped org state.
- IndexedDB sf-workspace: metadata cache, test sessions, screenshot data, all under org-scoped prefixes.
- sessionStorage: unsaved create/edit drafts for the current extension window. localStorage: last route, recent tools, and theme boot values.
- Exports: user-selected CSV/log/HTML/Markdown files. URLs are sanitized and text is redacted; optional email redaction defaults on.

Security assessment based on code

- Strong controls: same-instance URL check, credentials omitted from browser cookie handling in fetch, PKCE/state, org matching, source-tab pinning, no content scripts, no remote code, conservative access claims, production delete confirmation, optimistic concurrency.
- Residual risks: broad Salesforce host permissions and cookies permission are intrinsically sensitive; a browser SID grants the connected user's API authority while valid. IndexedDB screenshots and exported reports may contain business data. Local extension storage is only as protected as the Chrome profile/device.
- The extension is not a security boundary over Salesforce: server-side CRUD/FLS/sharing and API availability remain authoritative.

`src/manifest.json` - lines 17-45

`src/shared/api/client.ts` - lines 126-180

`PRIVACY.md` - lines 1-17

Build, installation, configuration, and testing

Prerequisites

- Chrome 116 or newer (manifest minimum).
- Node.js 20+ per README. The inspected environment used the repository lockfile and installed dependencies.

Build and load

Run the following from the repository root:

```
npm install
npm run build
npm test
```

- Open chrome://extensions, enable Developer mode, choose Load unpacked, and select the generated dist directory.
- Pin the extension. Click its toolbar icon or use Alt+Shift+S; this opens a dedicated workspace window tied to the launching tab. Chrome may also offer the declared side panel.
- npm run package rebuilds and creates release/salesforce-workspace-1.0.0.zip.

Salesforce configuration

- Normally, open a signed-in Salesforce page and launch the extension; auto-connect validates the tab session.
- For OAuth fallback, copy the callback URL shown in Settings, configure an External Client App/Connected App with the exact redirect URI, api plus refresh_token/offline_access scopes, and PKCE. Store only its Consumer Key in the extension.
- Per-org client ID and API version overrides are available under Edit org > Advanced. Use organization policy and permitted-user assignments appropriate to the target org.

Developer workflows

- npm run watch: incremental esbuild output.
- npm run preview: builds and serves a mock Chrome/Salesforce preview at localhost:5174; query options include page=record|home|none and connect=fail. Preview code is outside the production bundle.
- npm run typecheck: tsc --noEmit. npm run lint: ESLint. npm test: Vitest. npm run check: all gates plus build.

Verified results on 24 September 2026

- PASS: TypeScript typecheck.
- PASS: 13 test files, 145 tests.
- PASS: production esbuild bundle.
- FAIL: ESLint reports localStorage/window/document undefined and an unused catch binding in src/sidepanel/theme-boot.js. Therefore npm run check fails before reaching its test/build steps, although those commands passed when run separately.

```
package.json - lines 1-31
scripts/build.mjs - lines 1-47
scripts/package.mjs - lines 1-13
README.md - lines 5-42, 66-74
```

Known limitations and recommended next improvements

Current limitations verified in code

- No Code & Automation Explainer; no Apex class/trigger/Flow-definition source retrieval or explanation.
- Access Explainer is conservative but not a complete sharing/runtime simulator.
- Dependency results inherit MetadataComponentDependency beta gaps and the 2,000-row cap.
- Record prevalidation does not use UI API for record-type picklists, lookup filters, or page-layout requirements.
- Users paging uses SOQL OFFSET and direct memberships only.
- RecentlyViewed and SOSL behavior depend on Salesforce indexing and visibility.
- Local heuristic parsers may not recognize novel Flow, debug-log, or CLI formats.
- Data does not sync across Chrome profiles/browsers.
- Privacy policy includes a pre-publication placeholder.
- Repository-wide lint is currently red because theme-boot.js lacks browser-global lint configuration and has an unused catch variable.

Prioritized improvements

Priority	Recommendation	Why
P0	Fix the theme-boot.js lint context and unused catch binding; keep npm run check green in CI.	Restores a trustworthy release gate.
P0	Replace the privacy-policy placeholder with publisher contact/effective date before store publication.	Publishing readiness and user accountability.
P1	Add end-to-end extension tests for source-tab closure/origin change, OAuth renewal, production delete, and IndexedDB cleanup.	High-risk flows are broader than unit coverage.
P1	Use Salesforce UI API where feasible for record-type values, layout-required fields, and lookup constraints.	Reduces save-time validation surprises.
P1	Add explicit storage/export size management for screenshots and test sessions.	Controls device storage and sensitive evidence retention.
P2	If desired, design a clearly separate Code & Automation Explainer using Tooling/Metadata retrieval, permission checks, source-size limits, and deterministic provenance.	Avoids overstating the current debug/Flow-error parsers as source explanation.
P2	Add a generated API matrix and parser fixtures for new Salesforce/CLI versions.	Makes compatibility drift visible.

Glossary

ApexLog

Tooling API object containing Apex execution log metadata.

Chrome MV3

Chrome extension platform version using service workers and a restrictive content security model.

Connected App / External Client App

Salesforce OAuth client configuration. The extension uses its Consumer Key as a public client.

CRUD

Create, read, update, and delete permissions/operations.

Describe

Salesforce REST metadata describing objects, fields, relationships, and access flags.

FLS

Field-level security controlling field visibility/editability.

IndexedDB

Browser database used here for larger caches, sessions, and screenshots.

MetadataComponentDependency

Beta Tooling API object representing selected metadata dependencies.

My Domain

Organization-specific Salesforce hostname used to identify and connect to an org.

OAuth PKCE

Authorization Code flow protected by a per-request verifier/challenge; suitable for public clients.

Org-scoped storage

Keys prefixed by normalized org identity so data from one org is not reused for another.

REST sObject API

Salesforce endpoints used for object describe and record retrieval/changes.

SID

Salesforce browser session identifier read from the launching tab's cookie store and validated before use.

SOQL / SOSL

Salesforce Object Query Language / Salesforce Object Search Language.

Tooling API

Salesforce API for development and metadata-oriented objects such as ApexLog and CustomField.

UserRecordAccess

Salesforce object returning the specified user's access flags for a specific record.

Traceability note

Line references describe the inspected repository state and may move after edits. "Verified" means directly observable in code, configuration, tests, or a successful build; it does not certify behavior against every Salesforce org policy or license. No session token, credential, private org value, or secret is reproduced in this guide.

End of guide

Data Import & Export

IMPLEMENTATION ADDENDUM - supersedes the earlier feature inventory where the new top-level tab was absent.

Status: Working. Repository verification on 24 September 2026 found a separate top-level **Data Import & Export** sidebar section with **Import** and **Export** subtabs. It is intentionally separate from Records.

Component inventory

Component	Responsibility	Primary implementation
Route and sidebar	Top-level dataTransfer section; Import and Export tools; searchable route metadata.	src/sidebarpanel/state/routes.ts; src/sidebarpanel/App.tsx
Import UI	CSV upload, object/operation selection, auto/manual mapping, validation, preview, confirmation, progress, results.	src/sidebarpanel/modules/dataTransfer/DataImport.tsx
Export UI	Object/field selection, structured filter, preview, paged export, progress, cancel, CSV download.	src/sidebarpanel/modules/dataTransfer/DataExport.tsx
Transfer domain helpers	RFC 4180-style parsing, type conversion, mapping validation, safe identifiers, CSV encoding.	src/shared/salesforce/dataTransfer.ts
Salesforce client	REST Query, pagination, sObject create/update and external-ID upsert; abort and API limit handling.	src/shared/api/client.ts
Persistence/download	Org-scoped history metadata in IndexedDB; Blob URL downloads. No CSV payload persistence.	src/sidebarpanel/services/platform.ts; utils/download.ts

Cross-cutting data flow

Stage	Flow and control
Connection	ConnectionGate exposes the SalesforceClient and MetadataService for the explicitly selected org. The header identifies that org and user.
Authorization	Global/object describe limits selectable objects and writable fields. REST and Query APIs execute as the connected user; Salesforce enforces CRUD, FLS, sharing, restriction rules, validation rules, and automation.
Memory	Uploaded CSV text, parsed rows, previews, transfer records, and generated CSV remain component-memory state. Leaving the tool or changing org discards them.
History	Only timestamp, object, operation/field count, row counts, result counts, filter presence, and cancellation state are stored under the current org namespace.
Download	Results are generated locally as escaped CSV and downloaded through a temporary object URL.

Verification update

- **PASS:** TypeScript typecheck.
- **PASS:** 14 test files and 149 tests, including four CSV/mapping/validation tests.
- **PASS:** production esbuild output.
- **Known repository baseline:** ESLint still reports five pre-existing browser-global/unused-variable errors in src/sidebarpanel/theme-boot.js.

Import: components, validation, and writes

Status: Working. The write path cannot start until local validation succeeds and the user accepts a confirmation dialog.

User and data flow

- **1. Upload:** File input accepts CSV. The file is read with `File.text()`, parsed in memory, and capped at 10,000 data rows. Empty/duplicate headers and unclosed quotes are rejected.
- **2. Target:** Object choices come from global describe and are limited to createable/updateable objects. Insert, Update, and Upsert are explicit operations.
- **3. Mapping:** Headers auto-map by case-insensitive API name or label; each column may be remapped or ignored. Duplicate target fields are rejected.
- **4. Validation:** Conversion checks booleans, numbers, ISO dates, datetimes, text lengths, restricted picklists, writeability, required create fields, Id for Update, and the selected mapped external ID for Upsert.
- **5. Preview:** The first ten converted rows are shown before submission; validation issues include CSV row and field.
- **6. Confirmation:** The modal names the operation, object record count, and connected org/user, and warns that Salesforce rules and automation apply.
- **7. Execution:** Records are processed in client-side batches of 25 with bounded client concurrency. Insert uses `POST sObject`; Update uses `PATCH` by Id; Upsert uses `PATCH` by external-ID field/value.
- **8. Results:** Progress updates after each batch. Every row receives success/id or an error string. A downloadable row-level CSV contains row, success, id, and error.

Salesforce APIs and access enforcement

Purpose	API / endpoint	Enforcement
Discover objects	REST GET <code>/subjects/</code>	Object visibility and createable/updateable flags are those of the connected user.
Discover fields	REST GET <code>/subjects/{object}/describe/</code>	Mapping choices use createable/updateable/externalId and other describe attributes.
Insert	REST POST <code>/subjects/{object}/</code>	Salesforce applies object create access, FLS, requiredness, validation, triggers, Flows, and duplicate rules.
Update	REST PATCH <code>/subjects/{object}/{Id}</code>	Salesforce applies object update, record access/sharing, FLS, locks, validation, and automation.
Upsert	REST PATCH <code>/subjects/{object}/{externalIdField}/{value}</code>	Salesforce resolves the external ID and applies create or update authorization and record access.

Error handling and cancellation

- File and local validation errors block the confirmation button; Salesforce row failures do not stop unrelated rows.
- `AbortController` cancels outstanding fetches. Authentication refresh, typed API errors, network errors, and rate-limit usage tracking remain centralized in `SalesforceClient`.
- Import history is saved only after a completed run and contains aggregate metadata, never source rows or CSV contents.

Export: query, preview, progress, and cancellation

Status: Working. Export is a separate subtab and runs read-only queries under the connected Salesforce identity.

User and data flow

- **1. Target:** Object choices are limited to queryable objects returned by global describe.
- **2. Fields:** Users select readable described fields. Compound address/location containers are omitted; their concrete readable fields remain selectable where exposed.
- **3. Filter:** One structured field/operator/value predicate is supported. Object and field identifiers are allowlisted by describe and validated; string values are SOQL-escaped, numeric and Boolean values are type-checked. Raw SOQL is not accepted.
- **4. Preview:** A LIMIT 20 query displays records before a full export.
- **5. Export:** Query API pages are followed through nextRecordsUrl. Progress reports loaded versus total rows. The current implementation caps one export at 50,000 rows and stops early when known daily API use exceeds 90 percent.
- **6. Cancel:** Cancel aborts in-flight requests. Partial results are deliberately not offered as a download, and history records that the run was cancelled.
- **7. Download:** Completed rows are escaped into CSV locally and downloaded. The generated dataset remains in memory only.

Salesforce APIs, permissions, and record access

Concern	Implementation
Object and field access	Global/object describe supplies queryable objects and the connected user-visible field set.
Record access	REST Query API executes in the connected user context, so sharing, role hierarchy, teams/manual shares, restriction rules, and other platform access controls determine returned rows.
Pagination	GET /query starts the request; nextRecordsUrl is followed page by page by queryAll.
API protection	SalesforceClient caps concurrent requests, retries idempotent GETs on 502/503/504, tracks Sforce-Limit-Info, refreshes once after 401, and refuses cross-instance credential use.

Errors, privacy, and operational limits

- SOQL construction errors surface before the request; Salesforce and network errors use the shared error explanation UI.
- Export history is org-scoped aggregate metadata only. Record values, previews, and generated CSV are not written to extension storage.
- The current filter builder supports one predicate and six comparison operators. Multi-condition groups, Bulk API 2.0 jobs, resume-after-restart, and exports above 50,000 rows are **Planned / not implemented**.
- The import path uses REST record operations in batches of 25, not Bulk API 2.0. Server-side bulk job monitoring and restart persistence are **Planned / not implemented**.

Traceability

Tests: tests/dataTransfer.test.ts. UI: DataImport.tsx and DataExport.tsx. Core helpers: dataTransfer.ts. Client method: SalesforceClient.upsertRecord. Navigation: routes.ts and App.tsx. This addendum reflects the repository after implementation and supersedes any earlier statement in this guide that omits the Data Import & Export section.

Apex & Flow Explorer

IMPLEMENTATION ADDENDUM - top-level metadata explanation workspace.

Status: Working. The sidebar contains a dedicated **Apex & Flow Explorer** section with route-backed **Apex Classes**, **Apex Triggers**, and **Flows** subtabs. It reuses and expands the former Code & Automation Explainer.

Component and route inventory

Component	Responsibility	Implementation
Navigation	Top-level apexFlow section, three searchable tools, route history, deep link by explorerName.	src/sidepanel/state/routes.ts; src/sidepanel/App.tsx
Explorer UI	Responsive master-detail browser, search, Flow version picker, refresh, copy, source/metadata panel, handler links.	src/sidepanel/modules/development/CodeExplainer.tsx
Local analyzers	Apex/trigger structural parsing, line references, dependency detection, Flow element grouping and connector path.	src/shared/development/codeExplainer.ts
API and cache	Tooling queries and sObject reads through SalesforceClient; org-scoped definition cache in IndexedDB.	src/shared/api/client.ts; services/platform.ts
Shared UI	ConnectionGate, theme tokens, loading/error/empty states, buttons, pills, focus and responsive behavior.	src/sidepanel/components; src/sidepanel/styles.css

Common data flow and privacy

- **1. Select org:** Workspace context exposes the explicitly selected connected org. Changing org remounts the tool and changes the storage namespace.
- **2. List metadata:** Tooling Query returns only classes, triggers, and Flow versions visible to the connected Salesforce user.
- **3. Retrieve definition:** Tooling sObject GET retrieves Body for Apex or Metadata for the selected Flow version.
- **4. Explain locally:** Deterministic parsers inspect structural markers. No source or metadata is sent to an AI service or any non-Salesforce host.
- **5. Cache:** Results use code-explainer:{type}:{id}:{lastModified-or-version} inside the current org IndexedDB namespace. Refresh bypasses the cache.
- **6. Verify:** Users can inspect and copy the original source or metadata and compare it with cited methods, lines, or Flow element names.

Salesforce API surface

Purpose	Tooling API operation
List classes	SELECT Id, Name, Status, ApiVersion, LastModifiedDate FROM ApexClass ORDER BY Name
List triggers	SELECT Id, Name, Status, TableEnumOrId, ApiVersion, LastModifiedDate FROM ApexTrigger ORDER BY Name
List Flow versions	SELECT Id, MasterLabel, FullName, Status, VersionNumber, ProcessType, LastModifiedDate FROM Flow ORDER BY MasterLabel, VersionNumber DESC
Retrieve definition	GET /tooling/sobjects/ApexClass/{id}, ApexTrigger/{id}, or Flow/{id} under the negotiated REST API version.

Apex Classes and Apex Triggers

Apex Classes

- Search covers name and status. Selecting a class retrieves Body and shows it in a scrollable, copyable source panel.
- The summary identifies visible methods and inputs. Expandable sections cover execution markers, SOQL/read objects, DML/write markers, callouts and async work, explicit error handling, and dependencies.
- References include method names and one-based source line numbers for identifiable declarations and operations.
- Dependencies are inferred from class-qualified calls. Platform built-ins are filtered, but dynamic dispatch and reflection can hide relationships.

Apex Triggers

- The trigger declaration parser identifies the sObject and declared before/after events and summarizes when the trigger runs.
- SOQL, DML, callout/async, and error-handling sections reuse the Apex structural analyzer.
- Class-qualified references are presented as possible handler classes. Each handler is a link to the Apex Classes subtab using explorerName; retrieval still succeeds only when the user can access that class.
- If a declaration or behavior cannot be confidently recognized, the UI displays an Uncertain label and preserves the original source for verification.

Interpretation boundaries

Reported directly	Inferred or potentially incomplete
Names, status, API version metadata, trigger object/events, source text, explicit SOQL/DML/callout/try-catch markers.	Business purpose, effects hidden behind called code, dynamic SOQL, reflection, dependency injection, managed-package internals, runtime branch outcomes.
Method names and cited source lines.	A class-qualified reference is a likely dependency; it is not proof that the path executes in every transaction.

Error and permission behavior

- ConnectionGate blocks retrieval without a selected connected org. Salesforce authorization controls list and body visibility.
- Typed authentication, permission, network, cancellation, limit, and server errors use the shared ErrorAlert behavior. Refresh retries the selected definition.
- An inaccessible handler remains a navigation attempt that resolves to no accessible class rather than exposing hidden source.

Flows: versions, explanation, and visual path

Status: Working. Flows are grouped by name without discarding versions. Users explicitly select the version to retrieve and explain.

Version and metadata behavior

- The list shows Flow type and version count. The version picker shows version number, Active/Draft status, and ProcessType.
- The selected Flow record is retrieved from the Tooling API. The parser reads the returned Metadata object when present and preserves the full retrieved record otherwise.
- Expandable sections identify start conditions, input variables, decisions, assignments, record lookups/creates/updates/deletes, actions, subflows, screens, waits, and pauses by element name.
- Element counts and names are structural facts. Meaning that depends on formulas, expressions, runtime data, or unsupported elements is not stated as certain.

Visual path algorithm

- The renderer reads start.connector.targetReference and follows each supported element connector in order.
- Each node shows the Flow element name and collection type. faultConnector targets are shown on the originating node.
- Traversal is cycle-safe and capped at 100 connected nodes. Remaining recognized but unreachable elements are appended so they are not silently omitted.
- The diagram is intentionally a compact horizontal path, not a replacement for Flow Builder. Branch labels, complex loops, scheduled paths, and every outcome connector are not fully modeled.

Testing and verification

Check	Result
Static checks	TypeScript and ESLint pass.
Automated tests	17 files and 158 tests pass, including class parsing, trigger handler/timing, active and inactive Flow versions, and connector/fault path generation.
Production build	esbuild produces dist/sidepanel.js and dist/background.js.
Live-org dependency	End-to-end metadata coverage depends on the connected user permissions and the exact Tooling Metadata shape returned by that Salesforce API version.

Limitations

The explainer is deterministic and local, not semantic AI. It does not execute Apex or Flows, calculate runtime formulas, guarantee dependency completeness, or prove side effects hidden behind inaccessible code. Unsupported Flow elements remain visible in the raw metadata. The current visual path is connector-based and linearized; use Salesforce Flow Builder for the authoritative full graph.